Chapter 14

LLM Evaluation

Evaluation is the backbone of any rigorous machine learning pipeline, yet it is perhaps the most underappreciated component in the development of large language models. Unlike classical supervised learning, where a held-out test set with ground-truth labels provides a clean signal, evaluating LLMs requires grappling with open-ended generation, subjective quality judgments, multi-step reasoning chains, and the ever-present risk of benchmark contamination. This section provides a systematic treatment of the evaluation landscape: from the taxonomy of evaluation types and the mechanics of human annotation, through the mathematics of ranking metrics and the practicalities of LLM-as-judge, to the pitfalls that silently corrupt evaluation pipelines.

Why Evaluation is Hard for LLMs

Three fundamental challenges distinguish LLM evaluation from classical ML evaluation:

1. **Output space is unbounded.** A language model can produce any string; there is rarely a single correct answer.
2. **Quality is multidimensional.** Helpfulness, factuality, safety, coherence, and style are distinct axes that may trade off against each other.
3. **Evaluation is itself a language task.** Judging whether a response is good requires understanding, which means evaluation is susceptible to the same failure modes as generation.

14.1 Evaluation Scheme Design

Before collecting a single data point, practitioners must decide *what* to measure and *how* to measure it. A principled taxonomy prevents the common mistake of choosing metrics by convenience rather than by alignment with the deployment objective.

14.1.1 Taxonomy of Evaluation Types

Intrinsic vs. Extrinsic Evaluation. *Intrinsic* evaluation measures properties of the model output in isolation, without reference to a downstream application. Perplexity on a held-out corpus, BLEU score against reference translations, and pass@k on coding benchmarks are all intrinsic. *Extrinsic* evaluation measures the impact of the model on a real-world task or system: does integrating the LLM into a customer-service pipeline reduce ticket escalation rates? Does the coding assistant increase developer velocity?

The Intrinsic–Extrinsic Gap

Intrinsic metrics are cheap and reproducible but often poorly correlated with real-world utility. A model with lower perplexity is not necessarily more helpful. Extrinsic metrics are expensive and slow but directly measure what we care about. A mature evaluation strategy uses intrinsic metrics for rapid iteration and extrinsic metrics for final validation.

Automatic vs. Human Evaluation. *Automatic* evaluation uses deterministic functions (BLEU, exact match) or learned models (BERTScore, LLM-as-judge) to score outputs without human involvement. *Human* evaluation involves annotators rating or ranking model outputs. Table 14.1 summarises the trade-offs.

Table 14.1: Taxonomy of evaluation approaches with key trade-offs.

Type	Cost	Speed	Reproducibility	Validity
Automatic (rule-based)	Very low	Very fast	Perfect	Low–Medium
Automatic (model-based)	Low	Fast	High	Medium–High
Crowdsourced human	Medium	Days	Medium	Medium
Expert human	High	Weeks	Low–Medium	High
Extrinsic / A/B test	Very high	Months	Low	Very high

Reference-Based vs. Reference-Free Evaluation. Reference-based metrics (BLEU, ROUGE, BERTScore) compare model output to one or more gold-standard references. Reference-free metrics (perplexity, LLM-as-judge, human preference) assess quality without a reference. Reference-free approaches are essential when the output space is too large for exhaustive reference collection, as in open-ended dialogue.

14.1.2 When to Use What

Evaluation Strategy for a Dialogue Assistant

Development phase: Use automatic metrics (perplexity, ROUGE on summarisation sub-tasks, pass@k on tool-use) for rapid iteration. Run nightly benchmarks on standard suites (MMLU, HellaSwag, HumanEval).
Pre-release phase: Conduct a human preference study comparing the new model to the previous checkpoint. Use LLM-as-judge for scalable pairwise comparison on a diverse prompt set.
Post-release phase: Monitor extrinsic metrics (user satisfaction scores, task completion rates) and watch for distribution shift in production prompts.

A useful decision framework:

- If the task has a clear correct answer (math, code, factual QA): use exact match or execution-based metrics.
- If the task is open-ended but has reference outputs: use reference-based metrics as a lower bound, supplement with LLM-as-judge.
- If the task is subjective (helpfulness, tone, creativity): use human evaluation or a well-calibrated LLM judge.
- If the task involves multi-step agent behaviour: use task success rate and trajectory efficiency (Section 14.6).

14.2 Data Collection for Evaluation

High-quality evaluation data is the foundation of trustworthy benchmarks. This section covers the design of human annotation pipelines, statistical measures of annotation quality, and the choice between crowdsourcing and expert annotation.

14.2.1 Human Annotation Pipelines

A robust annotation pipeline consists of five stages:

1. **Task definition.** Specify the annotation task precisely: what is being rated, on what scale, and with what criteria. Ambiguity at this stage propagates into noisy labels.

2. **Guideline development.** Write annotation guidelines with worked examples covering edge cases. Iterate with a small pilot group before full deployment.

3. **Annotator recruitment and training.** Select annotators with appropriate background knowledge. Conduct a calibration session where annotators label the same examples and discuss disagreements.

4. **Quality control.** Embed gold-standard examples with known labels into the annotation queue. Flag annotators whose accuracy on gold examples falls below a threshold.

5. **Aggregation.** Combine multiple annotations per item using majority vote, averaging, or a probabilistic model (e.g., Dawid–Skene).

14.2.2 Inter-Annotator Agreement

Raw agreement (fraction of items where all annotators agree) is an inadequate measure because it does not account for chance agreement. Two standard chance-corrected measures are Cohen's κ [60] (two annotators) and Fleiss' κ [92] (multiple annotators).

Cohen's Kappa. Given two annotators labelling N items into k categories, let p_o be the observed agreement and p_e be the expected agreement under independence:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{14.1}$$

where

$$p_o = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\text{annotator 1 agrees with annotator 2 on item } i] \tag{14.2}$$

and

$$p_e = \sum_{c=1}^{k} p_{1c} \cdot p_{2c} \tag{14.3}$$

with p_{jc} being the proportion of items assigned to category c by annotator j. Cohen's κ ranges from -1 (perfect disagreement) through 0 (chance agreement) to 1 (perfect agreement). Values above 0.6 are generally considered acceptable; above 0.8 is strong agreement.

Fleiss' Kappa. For n annotators labelling N items into k categories, let n_{ij} be the number of annotators who assigned item i to category j. Define:

$$\bar{P}_i = \frac{1}{n(n-1)}\sum_{j=1}^{k} n_{ij}(n_{ij}-1), \qquad \bar{P} = \frac{1}{N}\sum_{i=1}^{N}\bar{P}_i \tag{14.4}$$

$$\bar{P}_j^e = \frac{1}{Nn}\sum_{i=1}^{N} n_{ij}, \qquad P_e = \sum_{j=1}^{k}\left(\bar{P}_j^e\right)^2 \tag{14.5}$$

$$\kappa_F = \frac{\bar{P} - P_e}{1 - P_e} \tag{14.6}$$

Kappa Limitations

Kappa is sensitive to the prevalence of categories: when one category dominates, kappa can be low even when raw agreement is high (the *kappa paradox*). For ordinal scales, weighted kappa (which penalises disagreements proportionally to their distance) is more appropriate. For LLM evaluation, where ratings are often on a 1–5 Likert scale, always report weighted kappa.

14.2.3 Annotation Guideline Design

Effective annotation guidelines share several properties:

- **Operationalised criteria.** Replace vague terms like "helpful" with concrete, observable behaviours: "The response directly addresses the user's question and provides all information needed to complete the stated task."
- **Worked examples.** Provide at least two examples per rating level, including borderline cases.
- **Decision trees.** For complex tasks, a flowchart that guides annotators through a sequence of binary decisions reduces cognitive load and improves consistency.
- **Explicit scope.** State what annotators should *not* consider (e.g., "Do not penalise for stylistic preferences; focus only on factual accuracy").

14.2.4 Crowdsourcing vs. Expert Annotation

Table 14.2: Comparison of crowdsourcing and expert annotation for LLM evaluation.

Dimension	Crowdsourcing	Expert Annotation
Cost per item	Low ($0.01 - -0.10$)	High ($1 - -50$)
Throughput	Very high	Low
Domain knowledge	Low	High
Consistency	Variable	High
Suitable tasks	Simple preference, fluency	Technical accuracy, safety
Platforms	MTurk, Prolific, Scale AI	Domain specialists, in-house
Quality control	Gold examples, attention checks	Calibration sessions, peer review

For safety-critical evaluation (e.g., detecting harmful outputs, evaluating medical advice), expert annotation is non-negotiable. For large-scale preference collection (e.g., building a reward model training set), crowdsourcing with rigorous quality control is often the only feasible option.

14.3 Synthetic Data Generation for Evaluation

Human annotation is expensive and slow. Synthetic data generation uses LLMs themselves to produce evaluation data at scale. This section covers the major paradigms.

14.3.1 LLM-as-Judge for Calibration

When using an LLM to generate evaluation labels, calibration is essential: the judge's scores must be aligned with human judgments. Let $h_i \in [0, 1]$ be the human preference score for item i and $\hat{h}_i$ be the judge's predicted score. Calibration error is measured by the Expected Calibration Error (ECE) [124]:

$$\text{ECE} = \sum_{b=1}^{B} \frac{|B_b|}{n} \left|\text{acc}(B_b) - \text{conf}(B_b)\right| \tag{14.7}$$

where B_b is the b-th confidence bin, $\text{acc}(B_b)$ is the fraction of items in the bin where the judge agrees with humans, and $\text{conf}(B_b)$ is the mean judge confidence in that bin.

A well-calibrated judge satisfies $\mathbb{E}[\hat{h}_i \mid \hat{h}_i = p] = p$ for all $p \in [0, 1]$. Calibration can be improved by temperature scaling: replacing the judge's raw logit z with z/T where T is tuned on a held-out calibration set to minimise negative log-likelihood.

14.3.2 Self-Instruct

Self-Instruct [371] bootstraps instruction-following data from a seed set of human-written tasks. The algorithm:

1. Maintain a task pool initialised with 175 seed tasks.
2. Sample 8 tasks from the pool; use them as few-shot examples to prompt the LLM to generate new tasks.
3. Filter generated tasks: remove near-duplicates (ROUGE-L similarity > 0.7 with any existing task), classify as classification vs. non-classification, and generate input–output instances.
4. Add accepted tasks to the pool.
5. Repeat until the desired pool size is reached.

Self-Instruct Prompt Template

```
system_prompt = """
Come up with a series of tasks:
Task 1: {seed_task_1_instruction}
Task 2: {seed_task_2_instruction}
...
Task 8: {seed_task_8_instruction}
Task 9:"""
```

The model completes the prompt, generating a new task instruction. A separate prompt then generates input–output pairs for the new task.

14.3.3 Evol-Instruct

Evol-Instruct [398] evolves a seed instruction set by iteratively rewriting instructions to be more complex or diverse. Two evolution operators are applied:

- **In-depth evolution:** Add constraints, increase reasoning steps, concretise abstractions, deepen domain knowledge requirements.
- **In-breadth evolution:** Generate a new instruction on a related but different topic, increasing topic diversity.

An instruction is accepted if it passes an elimination filter: the evolved instruction must not be a simple copy, must not contain "I'm sorry" or similar refusals, and must not be shorter than the original.

14.3.4 Constitutional AI Data Generation

Constitutional AI (CAI) [16] generates preference data by having the model critique and revise its own outputs according to a set of principles (the "constitution"). The pipeline:

1. **Supervised learning phase:** Sample a harmful prompt, generate an initial response, then prompt the model to critique the response according to a constitutional principle and revise it. Use the revised response as a supervised fine-tuning target.
2. **RL phase:** Generate pairs of responses (original vs. revised), use the model to label which is more constitutional, and train a preference model on these labels. Use the preference model as a reward signal for RLHF.

This approach generates preference data without human labelling of harmful content, reducing annotator exposure to distressing material.

14.3.5 Distillation for Evaluation Data

A powerful teacher model (e.g., GPT-4) can generate high-quality evaluation data for training a smaller judge model. The distillation pipeline:

1. Collect a diverse set of prompts and model responses.
2. Use the teacher to generate detailed judgments (scores + rationales).
3. Fine-tune a smaller model on (prompt, response, judgment) triples.
4. Validate the student judge against held-out human annotations.

Distillation Bias

A student judge distilled from a single teacher inherits the teacher's biases, including verbosity bias (preferring longer responses), self-enhancement bias (if the teacher is also the model being evaluated), and positional bias. Always validate distilled judges against independent human annotations.

14.3.6 Arena-Style Pairwise Generation

Chatbot Arena [433] generates evaluation data through a crowdsourced battle platform where users submit prompts and vote on which of two anonymised model responses they prefer. This produces a large-scale, naturally diverse dataset of pairwise preferences. The key design choices:

- **Anonymisation:** Model identities are hidden to prevent brand bias.
- **User-submitted prompts:** Ensures prompt diversity and real-world relevance.
- **Tie handling:** Users can declare a tie or indicate that both responses are bad.
- **Deduplication:** Near-duplicate prompts are filtered to prevent over-representation of common queries.

14.4 Metrics for Ranking Tasks

When the goal is to rank models by quality, pairwise comparison data is more reliable than absolute scores. This section derives the major ranking systems used in LLM evaluation.

14.4.1 ELO Rating System

The ELO system [84], originally developed for chess, assigns each player (model) a scalar rating R such that the expected score of player A against player B is:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}} \tag{14.8}$$

Derivation. The ELO model assumes that each player's performance on a given game is a random variable drawn from a logistic distribution centred at their rating. The probability that A beats B is:

$$P(A \succ B) = \sigma\left(\frac{R_A - R_B}{s}\right) = \frac{1}{1 + e^{-(R_A - R_B)/s}} \tag{14.9}$$

where $s = 400/\ln(10) \approx 173.7$ is a scale parameter chosen so that a 400-point difference corresponds to a $10:1$ odds ratio. After each game with outcome $S_A \in \{0, 0.5, 1\}$ (loss, draw, win), ratings are updated:

$$R_A \leftarrow R_A + K(S_A - E_A), \qquad R_B \leftarrow R_B + K(S_B - E_B) \tag{14.10}$$

where K is the K-factor controlling the learning rate. In Chatbot Arena, $K = 4$ is used.

ELO Intuition

ELO is a stochastic gradient descent update on the log-likelihood of the observed outcomes under the logistic model. Each game provides a noisy gradient signal; the K-factor controls the step size. A large K adapts quickly but is noisy; a small K is stable but slow to reflect true skill changes.

Bootstrap Confidence Intervals for ELO. Because ELO ratings depend on the order in which games are processed, confidence intervals are computed by bootstrap resampling: resample the battle log with replacement $B = 1000$ times, recompute ELO ratings from scratch for each resample, and report the 2.5th and 97.5th percentiles as the 95% confidence interval.

14.4.2 Bradley–Terry Model

The Bradley–Terry (BT) model [29] is a maximum-likelihood alternative to ELO. Given n models with strength parameters $\beta_1, \ldots, \beta_n > 0$, the probability that model i beats model j is:

$$P(i \succ j) = \frac{\beta_i}{\beta_i + \beta_j} \tag{14.11}$$

Given a set of pairwise outcomes $\{(i_k, j_k, y_k)\}_{k=1}^{M}$ where $y_k = 1$ if i_k beats j_k and $y_k = 0$ otherwise, the log-likelihood is:

$$\ell(\boldsymbol{\beta}) = \sum_{k=1}^{M} \left[y_k \log \frac{\beta_{i_k}}{\beta_{i_k} + \beta_{j_k}} + (1 - y_k) \log \frac{\beta_{j_k}}{\beta_{i_k} + \beta_{j_k}} \right] \tag{14.12}$$

The MLE $\hat{\boldsymbol{\beta}}$ is found by iterative scaling or gradient ascent. The BT model is identifiable only up to a multiplicative constant; a common normalisation is $\sum_i \log \beta_i = 0$. Working in log-space with $\theta_i = \log \beta_i$ gives:

$$P(i \succ j) = \sigma(\theta_i - \theta_j) \tag{14.13}$$

which is equivalent to a logistic regression with item-specific intercepts. The BT model is preferred over ELO when the full battle history is available, as it uses all data simultaneously rather than processing games sequentially.

14.4.3 TrueSkill

TrueSkill [138] is a Bayesian skill rating system that models each player's skill as a Gaussian random variable $s_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$. The performance of player i in a game is $p_i = s_i + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, \beta^2)$ is game-specific noise. Player i beats player j if $p_i > p_j$.

The posterior update after observing $i \succ j$ is computed via expectation propagation (EP). The key update equations for the winner are:

$$\mu_i \leftarrow \mu_i + \frac{\sigma_i^2}{c} \cdot v\left(\frac{\mu_i - \mu_j}{c}\right) \tag{14.14}$$

$$\sigma_i^2 \leftarrow \sigma_i^2 \left[1 - \frac{\sigma_i^2}{c^2} \cdot w\left(\frac{\mu_i - \mu_j}{c}\right)\right] \tag{14.15}$$

where $c = \sqrt{2\beta^2 + \sigma_i^2 + \sigma_j^2}$, and $v(t) = \phi(t)/\Phi(t)$, $w(t) = v(t)(v(t) + t)$ are the truncated Gaussian correction factors (ϕ and Φ are the standard normal PDF and CDF). TrueSkill's uncertainty estimate σ_i is particularly useful for identifying models that need more evaluation data.

14.4.4 Win Rate with Confidence Intervals

The simplest ranking metric is the win rate: the fraction of pairwise comparisons in which model A is preferred. Given n comparisons with w wins, the win rate is $\hat{p} = w/n$. A Wilson score confidence interval [380] is preferred over the naive Wald interval because it has better coverage near $p = 0$ and $p = 1$:

$$\text{CI} = \frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}} \tag{14.16}$$

where $z = 1.96$ for a 95% interval. For multi-way comparisons, win rate should be computed against a fixed baseline model to ensure comparability.

14.4.5 Chatbot Arena Methodology

Chatbot Arena [433] combines the above elements into a production-scale evaluation system:

1. Users submit prompts and receive responses from two anonymised models.
2. Users vote for the preferred response (or declare a tie).
3. Votes are aggregated using the BT model to produce a leaderboard.
4. Bootstrap confidence intervals are reported for each model's score.
5. Models with overlapping confidence intervals are considered statistically indistinguishable.

As of 2024, Chatbot Arena has collected over one million human preference votes, making it the largest publicly available LLM preference dataset.

14.5 Metrics for Generation Tasks

Generation metrics quantify the quality of model outputs for tasks with reference answers or well-defined correctness criteria.

14.5.1 BLEU

BLEU (Bilingual Evaluation Understudy) [283] measures n-gram precision between a hypothesis h and one or more references $\mathcal{R}$:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right) \tag{14.17}$$

where p_n is the modified n-gram precision, $w_n = 1/N$ are uniform weights, and BP is the brevity penalty:

$$\text{BP} = \begin{cases} 1 & \text{if } |h| > |r| \\ e^{1-|r|/|h|} & \text{if } |h| \leq |r| \end{cases} \tag{14.18}$$

with $|r|$ being the length of the closest reference. Modified n-gram precision clips each n-gram count to its maximum count in any reference:

$$p_n = \frac{\sum_{\text{ngram} \in h} \min(\text{count}(\text{ngram}, h),\ \max_{r \in \mathcal{R}} \text{count}(\text{ngram}, r))}{\sum_{\text{ngram} \in h} \text{count}(\text{ngram}, h)} \tag{14.19}$$

BLEU Limitations

BLEU was designed for machine translation with multiple references. For open-ended generation with a single reference, BLEU scores are often near zero even for high-quality outputs. BLEU does not capture semantic similarity, penalises valid paraphrases, and is sensitive to tokenisation. Use

BLEU only when multiple diverse references are available and the task has low output diversity.

14.5.2 ROUGE

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) [216] is a family of recall-oriented metrics designed for summarisation:

$$\text{ROUGE-N} = \frac{\sum_{r \in \mathcal{R}} \sum_{\text{ngram} \in r} \min(\text{count}(\text{ngram}, h), \text{count}(\text{ngram}, r))}{\sum_{r \in \mathcal{R}} \sum_{\text{ngram} \in r} \text{count}(\text{ngram}, r)} \tag{14.20}$$

$$\text{ROUGE-L} = \frac{\text{LCS}(h, r)}{|r|} \tag{14.21}$$

where LCS denotes the longest common subsequence. ROUGE-1 and ROUGE-2 measure unigram and bigram recall; ROUGE-L captures sentence-level structure. The F-measure variant balances precision and recall:

$$\text{ROUGE-N}_F = \frac{(1 + \beta^2) \cdot P \cdot R}{\beta^2 P + R} \tag{14.22}$$

with $\beta = 1$ for equal weighting.

14.5.3 BERTScore

BERTScore [426] computes token-level similarity using contextual embeddings from a pre-trained BERT model. Given hypothesis tokens $\hat{\mathbf{x}} = \langle \hat{x}_1, \ldots, \hat{x}_m \rangle$ and reference tokens $\mathbf{x} = \langle x_1, \ldots, x_n \rangle$ with embeddings $\hat{\mathbf{e}}_i$ and $\mathbf{e}_j$:

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{x_j \in \mathbf{x}} \max_{\hat{x}_i \in \hat{\mathbf{x}}} \frac{\hat{\mathbf{e}}_i^\top \mathbf{e}_j}{\|\hat{\mathbf{e}}_i\| \|\mathbf{e}_j\|} \tag{14.23}$$

$$P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{\hat{x}_i \in \hat{\mathbf{x}}} \max_{x_j \in \mathbf{x}} \frac{\hat{\mathbf{e}}_i^\top \mathbf{e}_j}{\|\hat{\mathbf{e}}_i\| \|\mathbf{e}_j\|} \tag{14.24}$$

$$F_{\text{BERT}} = 2 \cdot \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \tag{14.25}$$

BERTScore correlates better with human judgments than BLEU and ROUGE, particularly for paraphrases and semantically equivalent but lexically different outputs. Importance weighting using inverse document frequency (IDF) further improves correlation:

$$R_{\text{BERT}}^{\text{idf}} = \frac{\sum_{x_j \in \mathbf{x}} \text{idf}(x_j) \max_{\hat{x}_i} \cos(\hat{\mathbf{e}}_i, \mathbf{e}_j)}{\sum_{x_j \in \mathbf{x}} \text{idf}(x_j)} \tag{14.26}$$

14.5.4 METEOR

METEOR [19] addresses BLEU's recall blindness by computing an F-score over unigram matches, with additional modules for stemming and synonym matching:

$$\text{METEOR} = F_{\text{mean}} \cdot (1 - \text{Pen}) \tag{14.27}$$

where $F_{\text{mean}} = \frac{10PR}{R + 9P}$ (recall-weighted harmonic mean) and the fragmentation penalty $\text{Pen} = 0.5 \cdot (c/u_m)^3$ penalises non-contiguous matches (c = number of chunks, u_m = number of matched unigrams).

14.5.5 Perplexity

Perplexity measures how well a language model predicts a held-out text sequence $w_1, w_2, \ldots, w_T$:

$$\mathrm{PPL}(w_{1:T}) = \exp\left(-\frac{1}{T}\sum_{t=1}^{T}\log P_\theta(w_t \mid w_{1:t-1})\right) \tag{14.28}$$

Lower perplexity indicates better predictive performance. Perplexity is useful for comparing models on the same tokenisation and test set, but is not directly comparable across models with different vocabularies or tokenisers. For evaluation purposes, perplexity is most useful as a sanity check and for detecting distribution shift.

14.5.6 Pass@k for Code

For code generation, functional correctness is measured by executing generated code against test cases. The pass@k metric [47] estimates the probability that at least one of k generated samples passes all tests:

$$\text{pass@}k = \mathbb{E}_{\text{problems}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right] \tag{14.29}$$

where n is the total number of samples generated per problem and c is the number that pass. This unbiased estimator avoids the high variance of the naive estimator (which samples exactly k solutions and checks if any pass). In practice, $n = 200$ samples are generated and pass@1, pass@10, pass@100 are reported.

Pass@k Computation

```python
import numpy as np
from scipy.special import comb

def pass_at_k(n: int, c: int, k: int) -> float:
    """Unbiased estimator for pass@k.

    Args:
        n: total samples generated per problem
        c: number of samples that pass all tests
        k: number of samples to consider
    """
    if n - c < k:
        return 1.0
    return 1.0 - comb(n - c, k, exact=True) / comb(n, k, exact=True)

# Example: 200 samples, 15 pass, compute pass@1, pass@10, pass@100
for k in [1, 10, 100]:
    score = pass_at_k(n=200, c=15, k=k)
    print(f"pass@{k}: {score:.4f}")
# pass@1:   0.0750
# pass@10:  0.5391
# pass@100: 0.9999
```

14.5.7 Exact Match and F1

For extractive question answering (e.g., SQuAD), two standard metrics are:

- **Exact Match (EM):** Binary indicator of whether the predicted answer string exactly matches any gold answer after normalisation (lowercasing, removing articles and punctuation).
- **Token-level F1:** Treats prediction and gold answer as bags of tokens and computes the F1 score:

$$F1 = \frac{2 \cdot |\text{pred} \cap \text{gold}|}{|\text{pred}| + |\text{gold}|} \tag{14.30}$$

For multi-answer settings, the maximum F1 over all gold answers is reported.

Table 14.3: Summary of generation metrics: applicability and key properties.

Metric	Task	Reference-free?	Human correlation
BLEU	Translation	No	Low–Medium
ROUGE	Summarisation	No	Medium
BERTScore	General NLG	No	High
METEOR	Translation	No	Medium–High
Perplexity	LM quality	Yes	Low
Pass@k	Code generation	No (tests)	Very high
Exact Match	Extractive QA	No	Very high
Token F1	Extractive QA	No	High

14.6 Metrics for Agentic Tasks

Agentic LLMs operate in environments, take sequences of actions, and must complete multi-step tasks. Standard generation metrics are insufficient; agentic evaluation requires metrics that capture task completion, efficiency, and the quality of intermediate steps.

14.6.1 Task Success Rate

The primary metric for agentic tasks is the task success rate (TSR): the fraction of tasks for which the agent achieves the specified goal state:

$$\text{TSR} = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \mathbf{1}[\text{goal}(\tau) \text{ achieved}] \tag{14.31}$$

Goal achievement is typically verified by a deterministic oracle (e.g., checking database state, file system state, or test case execution). For tasks with partial credit, a graded success metric can be defined:

$$\text{TSR}_{\text{graded}} = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \text{score}(\tau) \in [0, 1] \tag{14.32}$$

14.6.2 Trajectory Efficiency

A successful agent should complete tasks with minimal unnecessary actions. Trajectory efficiency measures the ratio of the optimal trajectory length to the agent's actual trajectory length:

$$\eta = \frac{L^*}{L_{\text{agent}}} \tag{14.33}$$

where L^* is the length of the shortest successful trajectory (computed by an oracle or human expert) and L_{agent} is the number of actions taken by the agent. $\eta \in (0, 1]$ with $\eta = 1$ indicating optimal efficiency. For failed trajectories, $\eta = 0$.

A complementary metric is the *redundancy rate*: the fraction of agent actions that are not present in any optimal trajectory.

14.6.3 Tool-Use Accuracy

For agents that invoke external tools (APIs, code interpreters, search engines), tool-use accuracy measures the correctness of tool calls:

$$\text{TUA} = \frac{\#\text{ correct tool calls}}{\#\text{ total tool calls}} \tag{14.34}$$

A tool call is correct if (a) the correct tool is selected, (b) the arguments are valid, and (c) the call is made at the appropriate point in the trajectory. Partial credit can be assigned for correct tool selection with incorrect arguments.

14.6.4 Multi-Step Reasoning Accuracy

For tasks requiring chains of reasoning (e.g., multi-hop QA, mathematical problem solving), step-level accuracy measures the fraction of reasoning steps that are correct:

$$\text{SRA} = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \frac{1}{|S_\tau|} \sum_{s \in S_\tau} \mathbf{1}[s \text{ is correct}] \tag{14.35}$$

where S_τ is the set of reasoning steps in trajectory τ. Step correctness can be verified by a process reward model (PRM) or by human annotation.

14.6.5 SWE-bench Methodology

SWE-bench [169] evaluates LLMs on real-world software engineering tasks: given a GitHub issue description and the repository codebase, the model must generate a patch that resolves the issue. Evaluation proceeds as follows:

1. The model is given the issue description and relevant code context.
2. The model generates a patch (unified diff format).
3. The patch is applied to the repository.
4. The repository's test suite is executed; the task is successful if all tests pass.

The primary metric is **% Resolved**: the fraction of issues for which the generated patch passes all tests. SWE-bench Verified is a curated subset of 500 problems verified by human annotators to be solvable and unambiguous. SWE-bench Lite is a 300-problem subset designed for faster evaluation.

SWE-bench Key Statistics (as of 2024)

- **Full benchmark:** 2,294 tasks from 12 popular Python repositories.
- **Best open-source agent:** ∼43% resolved (SWE-bench Verified).
- **Human performance:** ∼87% resolved (with 15 minutes per task).
- **Evaluation cost:** ∼$0.25 per task for API-based models.

14.6.6 WebArena Methodology

WebArena [440] evaluates agents on realistic web navigation tasks in a sandboxed browser environment. The benchmark includes 812 tasks across five web applications (e-commerce, social forum, collaborative development, content management, and maps). Evaluation:

- **Functional evaluation:** The task outcome is verified by checking the application state (e.g., "Was the item added to the cart?", "Was the post created?").
- **URL-based evaluation:** For navigation tasks, the final URL is compared to the expected URL.
- **Program-based evaluation:** A custom evaluator script checks complex conditions (e.g., "Is the price less than $50?").

The primary metric is task success rate. Human performance is approximately 78%; state-of-the-art agents achieve approximately 35–45%.

14.7 LLM-as-Judge

LLM-as-judge [433] uses a capable LLM to evaluate the outputs of other (or the same) LLMs. This approach scales to large evaluation sets without human annotation and can provide detailed rationales for its judgments.

Table 14.4: Comparison of agentic evaluation benchmarks.

Benchmark	Domain	# Tasks	Eval Method	SOTA (%)
SWE-bench	Software engineering	2,294	Test execution	∼43
SWE-bench Lite	Software engineering	300	Test execution	∼50
WebArena	Web navigation	812	State/URL/program	∼40
ALFWorld [329]	Household tasks	3,553	Simulator state	∼90
AgentBench [229]	Multi-domain	1,091	Task-specific	∼45

14.7.1 Setup and Prompt Templates

The judge is presented with a prompt, one or more model responses, and an evaluation rubric. Three common formats:

Pointwise scoring. The judge assigns an absolute score to a single response:

Pointwise Judge Prompt

```
POINTWISE_PROMPT = """
You are an expert evaluator. Rate the following response on a scale
of 1-10 for helpfulness, accuracy, and clarity.

[Question]
{question}

[Response]
{response}

Provide your evaluation in the following format:
Reasoning: <step-by-step analysis>
Score: <integer from 1 to 10>
"""
```

Pairwise comparison. The judge compares two responses and selects the better one:

Pairwise Judge Prompt

```
PAIRWISE_PROMPT = """
You are an expert evaluator. Compare the two responses below and
determine which is better. Consider helpfulness, accuracy, and
depth of explanation.

[Question]
{question}

[Response A]
{response_a}

[Response B]
{response_b}

Output exactly one of: [[A]], [[B]], or [[C]] (tie).
Reasoning: <your analysis>
Verdict: <[[A]], [[B]], or [[C]]>
"""
```

Reference-guided scoring. The judge is provided with a reference answer and rates the response relative to it. This is particularly useful for factual tasks where the judge may not have reliable

knowledge.

14.7.2 Position Bias Mitigation

LLM judges exhibit *position bias*: a systematic preference for the response appearing in a particular position (first or last). This bias can be as large as 10–15 percentage points. Mitigation strategies:

1. **Swap augmentation:** Evaluate each pair in both orders (A vs. B and B vs. A). A consistent judgment is accepted; an inconsistent judgment is recorded as a tie.
2. **Calibration prompting:** Explicitly instruct the judge: "Your evaluation should not be influenced by the order in which the responses are presented."
3. **Ensemble judging:** Use multiple judges with different position orderings and aggregate their verdicts.
4. **Chain-of-thought forcing:** Require the judge to produce a detailed rationale before the verdict, which reduces reliance on superficial positional cues.

Verbosity Bias

LLM judges also exhibit verbosity bias: longer responses are systematically preferred, even when the additional content is irrelevant or repetitive. To mitigate this, instruct the judge to penalise unnecessary length and to focus on the quality of information rather than quantity. Alternatively, truncate responses to a fixed length before judging.

14.7.3 Multi-Judge Panels

A single judge may have systematic biases. A panel of judges from different model families provides more robust evaluations. Given J judges with verdicts $v_1, \ldots, v_J \in \{A, B, \text{tie}\}$, the panel verdict is determined by majority vote. The panel agreement rate is:

$$\text{Agreement} = \frac{1}{\binom{J}{2}} \sum_{i<j} \mathbf{1}[v_i = v_j] \tag{14.36}$$

For a three-judge panel, a unanimous verdict (all three agree) is treated as high-confidence; a 2–1 split as medium-confidence; and a three-way tie as low-confidence.

14.7.4 Agreement Metrics for LLM Judges

To validate an LLM judge, its verdicts are compared to human annotations on a held-out set. Key metrics:

- **Agreement rate:** Fraction of items where judge and human agree.
- **Cohen's κ:** Chance-corrected agreement (Equation **??**).
- **Spearman's ρ:** Rank correlation between judge scores and human scores, appropriate for ordinal ratings.
- **Kendall's τ:** Alternative rank correlation that is more robust to ties.

A judge is considered reliable if it achieves $\kappa > 0.6$ and agreement rate $> 80\%$ with human annotators on a representative sample.

14.7.5 G-Eval Framework

G-Eval [231] is a structured framework for LLM-based evaluation that uses chain-of-thought prompting and token probability weighting to produce more reliable scores. The framework:

1. **Generate evaluation steps:** Prompt the LLM to generate a detailed rubric for the evaluation task (e.g., "List the steps you would take to evaluate the coherence of a summary").
2. **Score with probability weighting:** For each score value $s \in \{1, 2, 3, 4, 5\}$, obtain the log-probability $\log P_\theta(s \mid \text{prompt, steps, response})$ from the judge model. The final score is the probability-weighted average:

$$\text{G-Eval score} = \sum_{s=1}^{5} s \cdot \frac{e^{\log P_\theta(s)}}{\sum_{s'=1}^{5} e^{\log P_\theta(s')}} \tag{14.37}$$

3. **Normalise:** Map the score to $[0, 1]$ by dividing by the maximum score.

G-Eval achieves higher correlation with human judgments than direct prompting, particularly for nuanced dimensions like coherence and consistency, because the probability weighting captures the judge's uncertainty rather than forcing a discrete choice.

Why G-Eval Works

Standard prompting asks the judge to output a single token (e.g., "4"), which discards the model's uncertainty. G-Eval reads the probability distribution over all score tokens, effectively computing the expected score under the judge's belief. This is analogous to using the mean of a posterior distribution rather than the mode.

14.8 Evaluation Pitfalls

Even carefully designed evaluation pipelines can produce misleading results. This section catalogues the most common failure modes.

14.8.1 Benchmark Contamination

Benchmark contamination occurs when evaluation data appears in the model's training set, either directly (verbatim inclusion) or indirectly (paraphrased or semantically similar content). Contaminated models achieve inflated scores that do not reflect true generalisation ability.

Detection methods:

- **n-gram overlap:** Compute the fraction of evaluation examples with high n-gram overlap (e.g., ROUGE-L > 0.8) with the training corpus.
- **Membership inference:** Use a membership inference attack to estimate the probability that each evaluation example was in the training set.
- **Canary strings:** Embed unique, randomly generated strings in evaluation examples and check if the model can complete them.
- **Temporal holdout:** Use evaluation data created after the model's training cutoff date.

Mitigation:

- Maintain a private test set that is never released publicly.
- Regularly refresh benchmarks with new examples.
- Report training data cutoff dates and decontamination procedures.

14.8.2 Overfitting to Benchmarks

Even without direct contamination, models can be implicitly optimised for specific benchmarks through repeated evaluation and hyperparameter tuning. This is a form of *adaptive overfitting*: the benchmark leaks information into model development decisions.

The Benchmark Lifecycle

A benchmark's utility degrades over time as the research community optimises for it. MMLU [137], once a challenging test of world knowledge, now has models achieving near-human performance, yet these models still fail on novel knowledge tasks. New benchmarks should be treated as temporary signal sources, not permanent ground truth.

14.8.3 Goodhart's Law in Evaluation

Goodhart's Law states: *"When a measure becomes a target, it ceases to be a good measure."* [112] In LLM evaluation, this manifests in several ways:

- **Reward hacking:** Models trained with RLHF learn to exploit the reward model rather than genuinely improving. A model may learn to produce verbose, confident-sounding responses that score highly on the reward model but are factually incorrect.
- **Metric gaming:** Models fine-tuned to maximise BLEU or ROUGE may produce outputs that score well on these metrics but are less useful to humans.
- **Judge gaming:** Models trained with LLM-as-judge feedback may learn the judge's biases (e.g., verbosity bias) rather than genuinely improving quality.

Defences Against Goodhart's Law

1. **Metric diversity:** Use multiple metrics from different families; a model that games one metric will likely not game all simultaneously.
2. **Held-out evaluation:** Maintain evaluation metrics that are not used in training or model selection.
3. **Human spot-checks:** Regularly sample model outputs for human review, independent of automated metrics.
4. **Adversarial evaluation:** Actively probe for failure modes that automated metrics miss.
5. **Extrinsic validation:** Periodically validate intrinsic metrics against extrinsic outcomes.

14.8.4 Additional Pitfalls

Prompt sensitivity. LLM performance can vary dramatically with small changes to the evaluation prompt (e.g., adding "Think step by step" or changing the answer format). Always report the exact prompt used and consider evaluating across multiple prompt variants.

Aggregation artefacts. Averaging scores across tasks with different difficulty levels and score distributions can produce misleading aggregate metrics. A model that excels at easy tasks but fails at hard tasks may have the same average score as a model with uniform performance.

Selection bias in human evaluation. Human evaluators are not a random sample of end users. Annotators on crowdsourcing platforms may have different preferences, cultural backgrounds, and domain knowledge than the target user population.

Evaluation–deployment mismatch. Evaluation prompts are often shorter, cleaner, and more well-formed than real user queries. A model that performs well on benchmark prompts may degrade significantly on the noisy, ambiguous, multi-turn conversations that occur in production.